

Template

Process tabular data

Process tabular data via the specified circuit.

Format the data and the corresponding output as a table.

Supports multiple input and multiple output dimensions.

Optional: For predictive networks, shift the output (predictions) to align with the input.

Requirements

```
<< "Circuits.m"
```

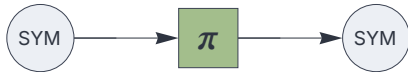
Circuit

```
dataflow = Import[NotebookDirectory[] <> "circuit.json", "Circuit"];
```

```
ExportString[dataflow, "Circuit"]
```

```
{
  "$schema": "https://creatingintelligence.org/schemas/circuit-v1.json",
  "hyperparameters": {"default": [3000, 10]},
  "dataflow": [
    {"component": "input", "plugin": "codec", "send": [11]},
    {"component": "temporal", "plugin": "permutation", "send":
      [21], "receive": [11], "capacity": 8, "decimate": 0.8},
    {"component": "output", "plugin": "codec", "receive": [21]}
  ]
}
```

```
circ = Circuit[dataflow];  
circ["schematics"][ImageSize→ 220]
```



Data

```
data = Import[ NotebookDirectory[] <> "data.csv"];
```

Run

```
out = List /@ circ["function"] @@@ data;
```

Format data and output side-by-side

```
ArrayToMarkdownTable[inputs_List, out_List, predictiveQ_ : False] :=
  Module[{outputs, combinedRaw, headerRow, allRows, colWidths, paddedTransposed},

    outputs = out;

    (* align input and output rows *)
    If[predictiveQ, outputs = RotateRight[outputs];
    outputs[[1]] = Null & /@ outputs[[1]];

    (* Convert all elements to strings *)
    combinedRaw = Map[ToString, Join[inputs, outputs, 2], {2}];

    (* Generate header row *)
    headerRow = Join[
      ConstantArray["in", Length[First[inputs]]],
      ConstantArray["out", Length[First[outputs]]]
    ];

    allRows = Prepend[combinedRaw, headerRow];

    (* Calculate maximum string length per column (minimum 3 for Markdown separator) *)
    colWidths = Max[3, #] & /@ Map[Max, Map[StringLength, Transpose[allRows], {2}]];

    (* Pad strings to match the max width of their respective columns *)
    paddedTransposed = MapThread[StringPadRight, {Transpose[allRows], colWidths}];
    paddedRows = Transpose[paddedTransposed];

    (* Generate separator row with correctly sized dash strings *)
    separatorRow = StringPadRight["", #, "-"] & /@ colWidths;

    (* Format a single row with Markdown pipes *)
    formatRow[row_] := "|" <> StringRiffle[row, " | "] <> " |";

    (* Insert separator between header and body, then format and join *)
    StringRiffle[
      formatRow /@ Insert[paddedRows, separatorRow, 2],
      "\n"
    ]
  ]
```

```
ArrayToMarkdownTable[data, out, True]
```

in	out
---	----
3	Null
1	Null
4	Null
1	Null
5	4
9	1
2	5
6	9
5	2
3	6
5	Null
8	3
9	5
7	5
9	9
3	2
2	9
3	8
8	2
4	3
6	8
2	5
6	6
4	2
3	6
3	5
8	8
3	9
2	8
7	3
9	3
5	3
0	9
2	5
8	6
8	4
4	3

1	6	
9	2	
7	9	
1	2	
6	3	
9	5	
3	3	
9	5	
9	8	
3	9	
7	2	
5	9	
1	3	
0	5	
5	9	
8	Null	
2	9	
0	6	
9	5	
7	3	
4	9	
9	3	
4	9	
4	3	
5	9	
9	9	
2	2	
3	6	
0	5	
7	3	
8	9	
1	9	
6	4	
4	4	
0	3	
6	9	
2	8	
8	3	
6	4	
2	4	
0	8	
8	9	

9	4	
9	3	
8	9	
6	9	
2	2	
8	3	
0	8	
3	2	
4	8	
8	8	
2	9	
5	9	
3	9	
4	3	
2	8	
1	8	
1	6	
7	9	
0	9	
6	9	
7	2	
3	8	
1	5	
4	4	
1	1	
5	5	
9	9	
2	2	
6	6	
5	5	
3	3	
5	5	
8	8	
9	9	
7	7	
9	9	
3	3	
2	2	
3	3	
8	8	
4	4	
6	6	

2	2	
6	6	
4	4	
3	3	
3	3	
8	8	
3	3	
2	2	
7	7	
9	9	
5	5	
0	0	
2	2	
8	8	
8	8	
4	4	
1	1	
9	9	
7	7	
1	1	
6	6	
9	9	
3	3	
9	9	
9	9	
3	3	
7	7	
5	5	
1	1	
0	0	
5	5	
8	8	
2	2	
0	0	
9	9	
7	7	
4	4	
9	9	
4	4	
4	4	
5	5	
9	9	

2	2	
3	3	
0	0	
7	7	
8	8	
1	1	
6	6	
4	4	
0	0	
6	6	
2	2	
8	8	
6	6	
2	2	
0	0	
8	8	
9	9	
9	9	
8	8	
6	6	
2	2	
8	8	
0	0	
3	3	
4	4	
8	8	
2	2	
5	5	
3	3	
4	4	
2	2	
1	1	
1	1	
7	7	
0	0	
6	6	
7	7	
3	3	
1	1	
4	4	
1	1	
5	5	

9	9	
2	2	
6	6	
5	5	
3	3	
5	5	
8	8	
9	9	
7	7	
9	9	
3	3	
2	2	
3	3	
8	8	
4	4	
6	6	
2	2	
6	6	
4	4	
3	3	
3	3	
8	8	
3	3	
2	2	
7	7	
9	9	
5	5	
0	0	
2	2	
8	8	
8	8	
4	4	
1	1	
9	9	
7	7	
1	1	
6	6	
9	9	
3	3	
9	9	
9	9	
3	3	

7	7	
5	5	
1	1	
0	0	
5	5	
8	8	
2	2	
0	0	
9	9	
7	7	
4	4	
9	9	
4	4	
4	4	
5	5	
9	9	
2	2	
3	3	
0	0	
7	7	
8	8	
1	1	
6	6	
4	4	
0	0	
6	6	
2	2	
8	8	
6	6	
2	2	
0	0	
8	8	
9	9	
9	9	
8	8	
6	6	
2	2	
8	8	
0	0	
3	3	
4	4	
8	8	

2	2	
5	5	
3	3	
4	4	
2	2	
1	1	
1	1	
7	7	
0	0	
6	6	
7	7	
3	3	
1	1	
4	4	
1	1	
5	5	
9	9	
2	2	
6	6	
5	5	
3	3	
5	5	
8	8	
9	9	
7	7	
9	9	
3	3	
2	2	
3	3	
8	8	
4	4	
6	6	
2	2	
6	6	
4	4	
3	3	
3	3	
8	8	
3	3	
2	2	
7	7	
9	9	

5	5	
0	0	
2	2	
8	8	
8	8	
4	4	
1	1	
9	9	
7	7	
1	1	
6	6	
9	9	
3	3	
9	9	
9	9	
3	3	
7	7	
5	5	
1	1	
0	0	
5	5	
8	8	
2	2	
0	0	
9	9	
7	7	
4	4	
9	9	
4	4	
4	4	
5	5	
9	9	
2	2	
3	3	
0	0	
7	7	
8	8	
1	1	
6	6	
4	4	
0	0	
6	6	

	2		2	
	8		8	
	6		6	
	2		2	
	0		0	
	8		8	
	9		9	
	9		9	
	8		8	
	6		6	
	2		2	
	8		8	
	0		0	
	3		3	
	4		4	
	8		8	
	2		2	
	5		5	
	3		3	
	4		4	
	2		2	
	1		1	
	1		1	
	7		7	
	0		0	
	6		6	
	7		7	